

confirmation_and_red

BOB-205 — DANGER_ROOTS scope hole: CONFIRMED + RED

Revision: 1 **Last modified:** 2026-08-26T00:00:00Z

Verdict: **CONFIRMED**, with three corrections and one larger unnamed gap.

1. The driver and its root list (verbatim)

scripts/pre_build_verification.sh:1481-1541 is invariant 39 (CM-DANGEROUS-COMBINATION-FAIL-CLOSED, ADVISORY). Line **1511**:

```
DANGER_ROOTS=(download-proxy/src plugins scripts qBitTorrent-go frontend/src)
```

Five roots. The loop (1513-1535) skips a root that does not exist (1514) and otherwise calls the gate once per root with `--root "${PROJECT_ROOT}/${_dr}" --quiet` (1517). The gate is invoked from exactly ONE place outside agent worktrees (scripts/pre_build_verification.sh:1507) — so a root absent from this array is scanned by **no arm**.

2. cmd/boba-ctl is absent — re-measured

metric	value
tracked .go files	4 (main.go, deploy_test.go, health_test.go, projectroot_test.go)
tracked .go LOC	947
non-test .go LOC	621 (main.go)
all tracked files	6 (adds go.mod 16, go.sum 23)

metric	value
all tracked LOC	986

The item's "4 files, 947 LOC" is **accurate** for the Go source. No drift.

3. Is it a §11.4.252 dangerous-combination surface? YES — 5 of 6 capabilities

§11.4.252 capability	evidence in cmd/boba-ctl/main.go
(1) mutation of shared resource	up/down/restart drive compose state — main.go:34,36, orchestrator main.go:95-105, compose.WithUpDetach main.go:121
(2) untrusted input	os.Args main.go:21,33,47,110,144,170,302; os.Getenv("PROJECT_ROOT") main.go:53; YAML host registry main.go:453-465
(3) credential access	SSH deploy auth selection main.go:432-440,496-509; KeyPath main.go:440
(4) external side effect	net.Dialer health probes main.go:239,323; SSH-driven remote boot main.go:430-431
(5) shell / exec	INDIRECT — delegated to digital.vasic.containers/pkg/ compose (main.go:13). No exec.Command in main.go itself.
(6) irreversible	down main.go:36,142

Correction to the item: it calls boba-ctl "a shell-exec plus state-mutation surface". State mutation is direct; shell-exec is one hop away through the Containers submodule. The ≥ 2 threshold is met several times over regardless.

Concrete fail-open-shaped construct found by review (not by the gate): main.go:498-509 authMethod() — default: return remote.AuthSSHKey (line 507). An unrecognised, typo'd or empty auth: value in the deploy host registry silently resolves to SSH-key auth instead of refusing. That is the §11.4.252 "default on ambiguous input" shape on a credential-selection path feeding an SSH remote deploy.

Honest qualification (§11.4.6): `main.go` contains `no _ = err / if err != nil {} / silent-return nil` shapes today (grep control-needed against `qBitTorrent-go/internal/config/config.go`, which does have them). So the scope hole here is currently **latent** — it hides no live gate-visible hit; it guarantees a future one lands unseen.

4. The FULL gap list

Rule for “first-party executable-source root” (reproducible):

a git-tracked path `P` where (a) `P` is not under any `.gitmodules` submodule path nor `constitution/ / superspec/`; (b) `P` holds ≥ 1 tracked file whose extension is in the gate’s **own** scannable set (`py go rs c cc cpp h hpp java cs js ts jsx tsx php rb`, gate line 318) — measuring against the gate’s own list keeps the comparison honest, because a `.sh`-only root is invisible for a *different* reason; (c) `P` is taken at top level, except where `DANGER_ROOTS` already uses a second-level path (`download-proxy/src`, `frontend/src`), in which case siblings of that path are reported separately.

4a. Gate-visible-language files OUTSIDE the scan scope

root	files	shipping?	concealed real hits (direct scan)
tests/	324	first-party, non-production	not scanned (excluded-by-intent, undeclared)
extension/	108	SHIPPED browser extension	0
docs/	51	non-shipping (research/qa assets)	not measured
challenges/	18	first-party validation harness	0
cmd/boba-ctl/	4	shipped orchestrator	0 (and Go-blind — see §5)
frontend/e2e/	3	test, sibling of frontend/src	not measured
tools/	1	first-party automation	3 (<code>plugin_update_automation.py:189,198</code> ,
<repo root>	1	webui-bridge.py, live host	1 (<code>webui-bridge.py:295</code>)

root	files	shipping?	concealed real hits (direct scan)
		service :7188	
frontend/ vitest.config.ts, frontend/ playwright.config.ts	2	config	not measured

Totals: 266 gate-visible first-party source files are inside DANGER_ROOTS; **512 are outside it** — i.e. **66% of the gate-visible first-party corpus is never scanned.** cmd/boba-ctl is 4 of those 512.

4b. The bigger unnamed gap — the repository root

webui-bridge.py (466 LOC) is an HTTP server on port 7188 (BaseHTTPRequestHandler :85, do_GET/do_POST :92-97, self.path :103, outbound urlopen :273) that reads qBittorrent connection env (:55-59,74). It is **not under any DANGER_ROOT**. Pointing the gate at it reports a real hit:

```
... silent default return (exception handler returns a trivial
literal with
no re-raise/log) at .../webui-bridge.py:295 (§11.4.252)
```

(webui-bridge.py:293-296 — except Exception: return False around urlparse. Whether that specific one is a real fail-open or a benign narrow guard is a triage question; the point is the gate **reports** it and invariant 39 **never sees** it.)

The repo root additionally holds **13 first-party shell scripts** including start.sh (1288 LOC — per CLAUDE.md *the* container orchestrator), stop.sh, setup.sh, install-plugin.sh, ci.sh, and the credential-handling init-qbit-password.sh / fix-qbit-password.sh. They are outside the scope **and** outside the language set (§4c).

Measured consequence: the scope hole conceals **at least 4 real hits the gate itself reports on sight** (1 at the repo root + 3 in tools/). Invariant 39's advisory "36 hits" is therefore a §11.4.201(6) under-count of the first-party population, not a census.

4c. A THIRD hole the item did not name — the LANGUAGE hole

Gate line 318: exts="\${DANGEROUS_COMBO_EXT:-py go rs c cc cpp h hpp java cs js ts jsx tsx php rb}". **sh is not in the list.** Measured: a .sh fail-open and a .py fail-open planted in the SAME

scanned root → py reported, sh **not reported**, and the gate emits **zero** mention of shell (grep -ciE 'shell|\.sh|bash' on its output = 0).

Consequence: scripts/ **is** a DANGER_ROOT but holds 71 .sh and only 3 .py + 1 .js — so 71/75 of that root is silently invisible while the driver prints “*no fail-open anti-pattern across N first-party source root(s)*”. Project-wide, **198 tracked shell files** are unscannable by language.

Correction to the item’s acceptance criterion (4): it states the gate “already implements correctly” an honest §11.4.3 SKIP-with-reason for unenumerated extensions. Measured: it does so for the **Python arm’s** degradations (missing interpreter / parse failure — gate lines 843, 863, 874, 876), but for an extension entirely absent from the ext list it says **nothing at all**. That is a silent false-null, not an honest skip.

5. BOB-205 vs BOB-191 — distinct, and they COMPOUND

Measured in one temp root containing both a Go needle and a Python needle:

```
go-needle hits: 0
py-needle hits: 1
```

The gate routes only *.py to its AST analyser (gate line 670); everything else falls to a line-based text scanner with no Go shape to match. **Go-blindness independently confirmed.**

And, pointing the gate **directly** at the real cmd/boba-ctl:

```
CM-DANGEROUS-COMBINATION-FAIL-CLOSED: PASS — no swallowed-
exception,
    silent-default-return or credential-default-to-literal anti-
patterns found
```

— a green verdict over 947 LOC it cannot analyse.

They are NOT the same defect, and fixing BOB-205 alone is not sufficient. Adding cmd/boba-ctl to DANGER_ROOTS moves it from *never looked at* to *looked at and falsely green* — a §11.4.201(6) FALSE-NULL, which is the worse of the two states because it now counts as covered. BOB-191 must land too.

6. The RED

```
tests/pre_build/test_bob205_danger_roots_scope.sh
```

Oracle (§11.4.245): METAMORPHIC + INVARIANT, with a control needle. Two **byte-identical** Python fail-open needles are planted in a hermetic fixture: one in a root that IS in DANGER_ROOTS (CONTROL), one in cmd/boba-ctl (PROBE). The scanner is driven exactly as invariant 39 drives it. Identical input in two locations must produce identical verdicts; the needles are byte-identical (sha256 asserted equal at runtime), so matcher strength, language support and gate version are all held constant and **scope is the only free variable**. The oracle is structurally independent of the code under test: the expected verdict comes from the metamorphic relation, not from reading DANGER_ROOTS.

Shape (a) — direct — was chosen over shape (b) — structural. (b) is included as a clearly-labelled SECONDARY assertion only; it pins list *content*, not *scanning*, so it would pass against a list entry that the loop never reaches (e.g. a typo'd path silently skipped by the `[[ -d ]]` guard at line 1514).

Confound avoided: the needle is **Python, not Go**, precisely because of BOB-191. A Go needle would stay unseen even after the scope fix, so the RED would never flip — a §11.4.201(1) FAIL-bluff. Fixing BOB-191 alone will **not** turn this test green; only widening the scope will.

Instrument viability (§11.4.201(7)(b)): if the CONTROL needle is not found the script exits **3 = ABORT**, not 1 = RED. A null from a blind instrument is not evidence.

Binding to the source of truth: the test parses DANGER_ROOTS from the real scripts/pre_build_verification.sh, so it flips GREEN the moment the root is added, with zero edits to the test.

It does **not** run pre_build_verification.sh and touches nothing outside its own mktmp -d.

RED (today) — red_run_live_gate.log, exit 1

```
gate sha256:
c5752428c08279029422384119f83364e29e7ca0a1262e21d2ea6c8e3a95b06a
DANGER_ROOTS (5): download-proxy/src plugins scripts qBitTorrent-
go frontend/src
needle sha256 (control == probe):
8d8bd909433bba614688f86d6973a54f0d39a42f6e4b8bcc89b10466f05b2e4e
roots scanned by the driver's own list: 5
CONTROL needle (download-proxy/src/): 1 hit(s) — instrument PROVEN
seeing
PROBE needle (cmd/boba-ctl/): 0 hit(s)
FAIL: a fail-open that IS reported when it sits in download-proxy/
src/ is INVISIBLE when it sits in cmd/boba-ctl/.
VERDICT: RED (primary=FAIL secondary=FAIL) — BOB-205 scope hole
reproduced
```

GREEN (polarity proof) — green_run_patched_driver.log, exit 0

Run against a **scratch copy** of the driver with `cmd/boba-ctl` appended (`B0B205_DRIVER=<scratch>`). The real `scripts/pre_build_verification.sh` was **not modified** (`git diff --stat` empty).

```
DANGER_ROOTS (6): ... frontend/src cmd/boba-ctl
CONTROL needle (download-proxy/src/): 1 hit(s) — instrument PROVEN
seeing
PROBE   needle (cmd/boba-ctl/): 1 hit(s)
VERDICT: GREEN (primary=PASS secondary=PASS)
```

Determinism (§11.4.50)

3/3 RED runs → exit 1. 3/3 GREEN runs → exit 0.

7. Fix direction — assessed, NOT applied

The item points at §11.4.251 (replace the hand-maintained list with a manifest derived from the build's own source of truth).
Assessment: feasible, and the source of truth already exists — but it must be an EXPLICIT manifest, not a derivation from the build graph.

Candidate sources of truth, measured:

1. **docker-compose.yml build contexts** — covers `download-proxy`, `qBitTorrent-go`, `submodules/jackett`; does **not** cover `plugins/`, `frontend/src`, `extension/`, `cmd/boba-ctl`, or `webui-bridge.py` (host process, no compose service). Insufficient alone.
2. **git + language markers** (`go.mod` / `package.json` / `pyproject.toml`) — discovers `cmd/boba-ctl` (has `go.mod`), `qBitTorrent-go`, `frontend`, `extension`; misses `plugins/` and `webui-bridge.py`, both of which have no module marker.
3. **Derive from git-tracked source extensions, minus a declared fence** — the only rule that reaches every gap above. This is the §11.4.224(E) exclusion-list shape: enumerate first-party source roots mechanically, and allow exclusion **only** via a checked-in list whose every entry is justified from the closed class set (generated / vendored / non-shipping-fixtures), with first-party exclusions additionally carrying a tracked §11.4.197 item.

Recommendation: option 3, i.e. flip the INCLUSION list into a *mechanically-derived* set plus a *declared exclusion fence*. That is what makes the invariant self-closing: a new root cannot exist without either being scanned or appearing, justified, in the fence. Option 1/2 leave the same class of hole in a different place.

If instead the operator keeps a hand-maintained list (a legitimate choice — it is auditable and cheap), the omission-catching guard is the missing piece: a gate that recomputes the derived root set and FAILs when a first-party source root exists that is neither in DANGER_ROOTS nor in the declared fence. That guard is the same computation as option 3 — so the honest framing is that the derivation must be built either way; the only question is whether it *drives* the scan or merely *audits* the hand list.

Whichever route: the tests/ question (324 files) is an operator decision per §11.4.66/§11.4.224(E) — production-only is a defensible scope, but it is currently **undeclared**, and an undeclared exclusion is what §11.4.224(E) forbids.

8. Not applied / out of scope for this task

No production fix was made. scripts/pre_build_verification.sh and constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh were read and (for the gate) snapshotted, never edited. pre_build_verification.sh was never executed.